



«Oye Quanela»

El modelo de palabra de activación de Quanela: cómo funciona, qué puede hacer, sus límites, cómo operarlo y cómo escalarlo.

0,4 %

activaciones perdidas sin ruido

1,2 %

perdidas con ruido de cocina (10 dB)

0

activaciones falsas en 17,3 h de audio

≈ 5 MB

descarga única por equipo

Documento técnico y operativo · Versión del modelo **oye-quanela-v1** · 30 de septiembre de 2026

Decisión de arquitectura: ADR 0016 (docs/adr/0016-oye-quanela-palabra-de-activacion.md) · Código de entrenamiento: ml/wake-word/ · Motor en la app: src/shared/voice/wakeWord/

Estado: **beta**. Evaluado con voces sintéticas y voces clonadas de hablantes reales; falta la prueba con personas en una cocina real (fase 2b).

Contenido

- 1 Resumen ejecutivo
- 2 Qué es una palabra de activación y por qué «Oye Quanela»
- 3 Cómo funciona: del micrófono a la acción
- 4 Las tres piezas del modelo
- 5 Cómo se entrenó
- 6 Capacidades medidas
- 7 Limitaciones y riesgos
- 8 Integración en Quanela: control, interfaz y privacidad
- 9 Rendimiento y costos
- 10 Cómo escala
- 11 Operación: ajustar, actualizar, revertir, diagnosticar
- 12 Cómo mejorarlo: hoja de ruta
- 13 Licencias y cumplimiento
- A Anexo: tabla completa de umbrales
- B Anexo: archivos, comandos y glosario

Cómo leer este documento

Las secciones 1–3 explican el sistema a cualquier persona. Las secciones 4–9 dan el detalle técnico y las cifras. Las secciones 10–12 responden a «¿qué pasa cuando crecemos?» y «¿cómo lo mantenemos?». Los anexos sirven de referencia rápida.

1. Resumen ejecutivo

En la pantalla de Cocina, quien cocina puede decir «**Oye Quanela**» y a continuación el comando (por ejemplo, «pedido dos mil cuarenta listo») sin tocar la tableta. Suena un tono corto y Quanela escucha el comando durante 5 segundos con el mismo reconocimiento de voz que ya usa el botón del micrófono.

Detectar la frase lo hace un **modelo pequeño propio** que corre **dentro del navegador de cada equipo**. El audio nunca sale del equipo, no se guarda y no cuesta nada por uso: no hay servidores de voz ni facturas por minuto.

Lo que ya hace bien

- Detecta la frase el 99,6 % de las veces sin ruido y el 98,8 % con ruido de cocina moderado.
- No se activó ni una vez en 17,3 horas de audio sin la frase: habla en español, cocina, música, ruido y habla en inglés.
- No confunde las palabras típicas de una cocina: canela, cazuela, Manuela, candela, ventana, «oye nena».
- Responde al terminar la frase, en menos de 0,1 s.
- Corre en el equipo con 4–6 % de un núcleo de un Mac moderno.

Lo que falta o hay que vigilar

- Se entrenó y se evaluó con voces sintéticas. Falta probarlo con personas reales en una cocina (fase 2b).
- Con ruido muy fuerte (5 dB) pierde alrededor del 4–10 % de las frases.
- Unas pocas frases muy parecidas todavía lo activan: 4 de 1.600 («Oye, cuarenta», «Oye, panela», «Quanela» sola).
- El micrófono queda abierto: gasta batería, y en iOS la pantalla debe quedar encendida.
- Falta medir el consumo en una tableta modesta.

Decisiones que lo definen

Decisión	Elección	Por qué
Frase	«Oye Quanela»	Es larga y poco común: menos activaciones falsas que decir solo «Quanela».
Tecnología	Arquitectura openWakeWord, con modelos exportados por nosotros	Es código abierto (Apache 2.0) y permite entrenar frases nuevas con voz sintética. Evitamos sus modelos preentrenados, que son de uso no comercial.
Dónde corre	En el navegador (onnxruntime-web)	Privacidad, costo cero por uso y funciona sin internet una vez descargado.
Control	Función <code>voice_wake_word</code> más un interruptor por equipo, apagado por defecto	La plataforma, el plan y la organización deciden quién la tiene. En cada tableta, alguien decide abrir el micrófono.
Umbral	0,9 con 2 tramos seguidos	Fue el mejor balance en la evaluación: 0 activaciones falsas y pocas frases perdidas. La plataforma lo puede cambiar sin tocar código.

2. Qué es una palabra de activación

Una palabra de activación (en inglés, *wake word*) es una frase fija que un equipo escucha de forma continua para «despertar». Es lo mismo que hacen «Oye Siri» u «OK Google». El detector **no entiende el idioma**: solo responde a una pregunta por cada fragmento de audio: ¿*acaba de sonar «Oye Quanela»*? Por eso puede ser pequeño, rápido y correr todo el tiempo sin enviar nada a internet.

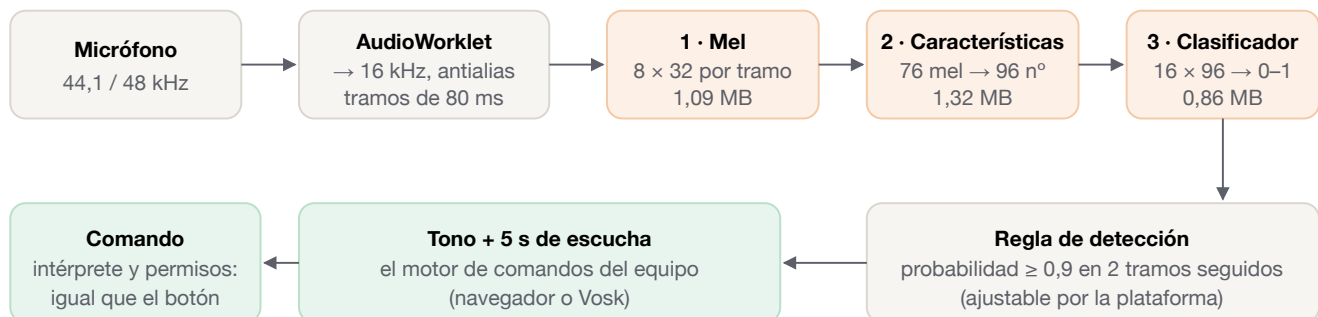
Dos modelos, dos trabajos

	Palabra de activación («Oye Quanela»)	Reconocimiento de comandos
Qué hace	Decide si se dijo la frase	Convierte a texto «pedido dos mil cuarenta listo»
Cuándo corre	Siempre, mientras manos libres está activo	Solo 5 s después de la frase, o al tocar el micrófono
Motor	Modelo propio (este documento)	El del navegador o Vosk sin internet (ADR 0015)
Error típico	Activarse sin que nadie lo llame, o no oír la frase	Entender mal el número o la acción
Consecuencia de un error	Se abren 5 s de escucha y no pasa nada si nadie habla	Protegida: los mismos permisos que los botones, y cancelar por voz desactivado con Vosk

Por qué «Oye Quanela»

- **Largo y distintivo:** tiene 5 sílabas y un sonido poco frecuente («kwa-NE-la»), así que es difícil que aparezca por casualidad en una conversación o en la radio.
- **Natural en español:** «Oye...» es como la gente llama a alguien en una cocina colombiana.
- **Evita confusiones con palabras de cocina:** panela, canela, cazuela, Manuela y candela suenan parecido. Por eso se usaron a propósito como ejemplos negativos al entrenar (sección 5).

3. Cómo funciona: del micrófono a la acción



Pausas automáticas

- Mientras se escucha o procesa un comando, la detección suelta el micrófono y lo retoma después.
- Mientras Quanela habla (y 0,7 s después) se descarta el audio. Un toque en el indicador lo pausa.

Paso a paso

1. **Captura.** El navegador entrega el audio del micrófono a su frecuencia nativa (44,1 o 48 kHz). Un *AudioWorklet* (código que corre en el hilo de audio) lo filtra para eliminar lo que está por encima de 7,2 kHz y lo convierte a 16 kHz. Lo entrega en tramos de 1.280 muestras (80 ms). Se pide cancelación de eco y control de ganancia, pero no supresión de ruido, porque el modelo se entrenó con audio sin procesar.
2. **Espectrograma mel.** Cada tramo, junto con los 30 ms anteriores, se convierte en 8 «fotos» del sonido de 32 bandas de frecuencia, entre 60 y 3.800 Hz, en escala logarítmica, como lo percibe el oído.

3. **Características.** Con las últimas 76 fotos (unos 0,8 s), una red convolucional de Google produce 96 números que resumen «cómo suena» ese momento. Hay un vector nuevo cada 80 ms.
4. **Clasificador.** Una red pequeña mira los últimos 16 vectores (unos 2 s de contexto) y responde con una probabilidad entre 0 y 1 de que se acabe de decir «Oye Quanela».
5. **Regla de detección.** Hace falta superar el umbral (0,9) en 2 tramos seguidos, es decir, sostenerse 160 ms, para evitar picos aislados. Tras detectar, el micrófono se libera y la escucha se reinicia desde cero. Así, la misma frase no puede activarlo dos veces.
6. **Encadenado.** Suena un tono de dos notas y se abre el reconocimiento de comandos durante 5 s. Si nadie habla, vuelve a esperar la frase sin mostrar error. Si hay comando, sigue el flujo de siempre: intérprete, validaciones, la misma mutación que el botón y la respuesta hablada.

Calentamiento: al empezar a escuchar, o tras una pausa, el detector necesita unos 2 s de audio (25 tramos) antes de dar su primera probabilidad. En la práctica es imperceptible: el indicador muestra «Preparando...» y luego «Oye Quanela».

4. Las tres piezas del modelo

Pieza	Entrada → salida	Tamaño	Origen y licencia	¿Se reentrena?
Espectrograma mel mel_spectrogram.onnx	1.760 muestras → 8 × 32	1,09 MB	Construido por nosotros con fórmulas estándar: ventana Hann de 400 muestras, FFT de 512 y banco mel Slaney. Recorte de 80 dB por clip.	No: es matemática fija.
Red de características speech_embedding.onnx	76 × 32 → 96	1,32 MB	Pesos de Google <code>speech_embedding/1</code> (Apache 2.0), convertidos por nosotros a ONNX: 19 convoluciones con normalización incorporada y 5 capas de reducción.	No: está congelada y se comparte entre frases.
Clasificador «Oye Quanela» oye_quanela.onnx	16 × 96 → probabilidad	0,86 MB	Propio: red densa 1.536 → 128 → 128 → 1 con LayerNorm y ReLU. Al entrenar lleva <i>dropout</i> 0,3 y ruido en las entradas.	Sí: es lo único que se entrena para cada frase.

Por qué exportamos nosotros los dos primeros modelos

El proyecto openWakeWord incluye estas dos piezas ya listas, pero sus modelos preentrenados se publican con una licencia no comercial. Para no heredar esa ambigüedad:

- el espectrograma se reescribió a partir de fórmulas estándar;
- la red de características se reconstruyó capa por capa a partir de los pesos originales de Google, que son Apache 2.0.

Luego se verificó que dieran lo mismo que los de openWakeWord.

Verificación	Diferencia máxima	Lectura
Espectrograma propio vs. openWakeWord	$1,5 \times 10^{-5}$	Idéntico a efectos prácticos (redondeo de coma flotante).
Red de características propia vs. openWakeWord	0,0	Bit a bit igual.
Las dos encadenadas, sobre una escala de ~63	$3,4 \times 10^{-5}$	Idéntico.
Cálculo por tramos propio vs. el de openWakeWord	0,0	Misma aritmética en <i>streaming</i> .
Navegador (onnxruntime-web) vs. Python, con el modelo final	$1,8 \times 10^{-7}$	La meta era menos de 0,01.

Una ventaja estructural: la base es compartida

Las piezas 1 y 2 son iguales para cualquier frase. Por eso:

- agregar otra frase («Oye Quanela, pausa» o una frase por marca) solo exige **entrenar otro clasificador** de menos de 1 MB;
- en el equipo, varias frases comparten el mismo cálculo pesado, y cada clasificador extra cuesta décimas de milisegundo por tramo (sección 10).

5. Cómo se entrenó

No existían grabaciones de gente diciendo «Oye Quanela», así que el clasificador se entrenó sobre todo con **voz sintética**: miles de voces artificiales diciendo la frase, mezcladas con ruido real. También aprendió con muchos ejemplos de lo que **no** es la frase.

Datos

Grupo	Qué contiene	Cantidad
Positivos	«Oye Quanela» en 11 formas escritas (con y sin coma, exclamación, pregunta; «cuanela», «kuanela»). 6.000 son de Kokoro, con 3 voces españolas mezcladas con 24 voces de otros idiomas y velocidades de 0,8 a 1,25. Otras 1.500 son de Piper <code>carl_fm</code> , con variación de entonación.	7.500 clips × 3 aumentos = 22.476
Frases parecidas	57 frases trampa: «Oye, panela», «Oye, canela», «Oye, cazuela», «Oye, Manuela», «Cuanela» sola, «Oye, cuarenta», «Oye, Juanita», «Venezuela», «Hay panela en la cazuela»...	5.542 clips × 2
Frases dirigidas	Las 31 frases que aún activaban el modelo a mitad del proceso	1.500 clips × 3
Habla sintética	Comandos de cocina y frases generales, dichas por las mismas voces sintéticas, para que «voz sintética» no se convierta en pista de la frase	2.129 clips × 2
Audio real sin la frase	FLEURS en español latinoamericano (8,2 h, el 60 % con ruido de cocina encima), habla de MUSAN (16,3 h), música de MUSAN (11,8 h), ruido de MUSAN (4,9 h) y cocina sintética (2,0 h)	43,2 h

Aumentos: cómo se simula una cocina real

Cada clip se transforma al azar antes de convertirlo en características:

- velocidad y tono de ± 7 %;
- reverberación de salas simuladas pequeñas y medianas (50 %);

- micrófono de banda limitada (30 %);
- nivel de voz de -42 a -14 dBFS, es decir, de lejos a cerca.

Luego se mezcla con un fondo con una relación señal/ruido de 0 a 25 dB; un 20 % queda casi limpio. Los fondos son:

- cocina sintética: campana, fritura, golpes metálicos, pitidos y agua;
- ruidos, música y habla de fondo.

La frase se coloca de modo que termine en los últimos 0,3 s de la ventana de 2 s, que es el momento en que el detector debe dispararse.

Entrenamiento

- Pérdida de entropía cruzada. Los ejemplos negativos pesan cada vez más a lo largo del entrenamiento (de 1 a 20 veces), porque una activación falsa es el error más molesto.
- **Minería de negativos difíciles:** tras 10.000 pasos se busca en todo el audio real qué fragmentos aún dan puntaje alto. Con ellos se hacen 5.000 pasos más, con un peso de 30.
- La regularización (*dropout* 0,3, ruido de $0,3 \sigma$ en las entradas y un decaimiento de pesos de 0,05) fue decisiva contra el sobreajuste.
- Se entrenó en un Mac con Apple Silicon: unos 30–40 min por ronda. La generación de datos y el cálculo de características tomaron varias horas.

Cuatro rondas hasta el modelo publicado

Ronda	Cambio	Frases parecidas que activan	Activaciones falsas
1	Base, sin regularizar	2,2 %	Hasta 28 por hora en ruido (aún no lo había visto)
2	Con <i>dropout</i> y ruido en las entradas	0,9 %	0 en 13 h
3	Frases dirigidas y música	0,19 %	0,12 por hora (en habla en inglés)
4 • publicada	Con habla de MUSAN	0,25 %	0 en 17,3 h

6. Capacidades medidas

La evaluación simula lo que pasa en la tableta: el audio se procesa **tramo a tramo, en *streaming***, con la misma regla de detección que la app. Todo el material de evaluación quedó **apartado** del entrenamiento:

- 8 voces de Kokoro que no se usaron para entrenar;
- 102 frases con voces clonadas (Chatterbox) de **hablantes reales** de FLEURS test;
- ruidos y salas distintos;
- 17,3 h de audio sin la frase.

99,6 %

detecta sin ruido

98,8 %

detecta con ruido de 10 dB

95,6 %

detecta con ruido de 5 dB

0 / 17,3 h

activaciones falsas

Métrica (umbral 0,9, 2 tramos)	Resultado	Meta de la ADR	Estado
Frases perdidas: sin ruido / 10 dB / 5 dB (502 frases)	0,4 % / 1,2 % / 4,4 %	≤ 5 % / ≤ 15 %	Cumple
Solo voces reales clonadas (102): sin ruido / 10 dB / 5 dB	2,0 % / 2,0 % / 9,8 %	—	Referencia
Activaciones falsas en 17,3 h (habla en español 2,9 h, cocina 0,5 h, música 8,7 h, ruido 1,2 h, habla en inglés 3,9 h)	0 (menos de 0,06 por hora)	≤ 0,5 por hora	Cumple
Palabras de cocina: canela, cazuela, Manuela, candela, ventana, «oye nena»	0 activaciones	Ninguna	Cumple
Todas las frases parecidas (1.600 intentos)	4 (0,25 %)	Ninguna	Casi
Latencia desde el final de la frase	Mediana −119 ms · p90 +60 ms	≤ 300 ms	Cumple
Paridad entre navegador y Python	$1,8 \times 10^{-7}$	< 0,01	Cumple
Descarga por equipo	3,26 MB de modelos + ~2,3 MB de runtime (brotli)	≤ 5 MB	En el límite

Las 4 frases parecidas que aún lo activan

«Oye, cuarenta» (1 de 24), «Oye, panela» (1 de 30), «Quanela» (1 de 30) y «¡Cuanela!» (1 de 32). Todas comparten el arranque «kwa» o la terminación «-nela». El costo es bajo: se abren 5 s de escucha y, si nadie da un comando válido, no pasa nada.

Latencia negativa: ¿cómo puede detectar antes de que termine?

La latencia se mide desde el final del clip, *incluida* la cola de reverberación. Una mediana negativa significa que suele dispararse durante la última sílaba («...la»). Es un buen comportamiento: el tono suena justo cuando la persona termina de hablar.

Qué pasa al mover el umbral

Subir el umbral reduce las activaciones falsas y las frases parecidas, a cambio de perder más frases dichas de verdad. El anexo A trae la tabla completa. Los puntos más útiles:

Configuración	Perdidas sin ruido	10 dB	5 dB	Falsas por hora	Frases parecidas
0,5 · 1 tramo (muy sensible)	0,2 %	0,4 %	2,0 %	0,58	22
0,8 · 2 tramos	0,2 %	1,2 %	3,8 %	0	8
0,9 · 2 tramos (publicado)	0,4 %	1,2 %	4,4 %	0	4
0,97 · 2 tramos	0,6 %	2,2 %	6,4 %	0	3
0,99 · 3 tramos (muy estricto)	2,0 %	5,4 %	12,0 %	0	1

7. Limitaciones y riesgos

Limitación	Qué significa	Mitigación actual	Qué haría falta
Sin voces humanas reales al entrenar	Las cifras vienen de voces sintéticas y clonadas. Con personas reales puede perder más frases, sobre todo con acentos, prisa o gritos.	Evaluación con voces clonadas de hablantes reales (2 % pérdidas sin ruido).	La fase 2b: 10–20 personas y 1–2 h de cocina, con consentimiento. Después, reentrenar con ese audio.
Ruido muy fuerte	Con 5 dB (extractor a tope, licuadora cerca) pierde entre el 4 y el 10 % de las frases.	Aumentos con ruido de 0 a 25 dB.	Hablar más cerca del equipo, o usar un micrófono externo. Grabaciones reales de la cocina para entrenar.
Frases muy parecidas	0,25 % de activaciones con «Oye, cuarenta», «Oye, panela» o «Quanela» sola.	Negativos dirigidos. Una activación falsa solo abre 5 s de escucha.	Subir el umbral o reentrenar con más negativos reales.
No reconoce quién habla	Cualquiera puede activarlo, incluida una voz en la radio o la televisión que diga la frase.	Los comandos pasan por los mismos permisos que los botones; cancelar por voz está desactivado con Vosk.	No es un control de seguridad y no debe usarse como tal.
Una sola frase	Solo «Oye Quanela». No responde a «Hola Quanela» ni a variantes.	—	Entrenar otro clasificador (sección 10).
Distancia	Se probó con niveles de voz equivalentes a hablar de cerca y a unos metros, en salas simuladas.	Aumentos de nivel y de reverberación.	Medir en una cocina real a 1, 2 y 4 m.
Micrófono siempre abierto	Consume batería y CPU. En iOS y Safari, el micrófono se corta al bloquear la pantalla.	Apagado por defecto, activación por equipo y pausa con un toque.	Dejar la tableta en modo quiosco con la pantalla encendida y medir la batería.
Navegadores	Necesita AudioWorklet, WebAssembly y getUserMedia (Chrome, Edge, Safari 14.1+ y Firefox actuales).	Si no hay soporte, el interruptor aparece desactivado.	—
Primera descarga	La primera vez descarga unos 5 MB (de 1 a 10 s según la red).	Descarga con progreso y caché de 1 año.	—
Voz propia de otros equipos	El eco de la propia tableta se descarta, pero si otra tableta cercana dice «Quanela...», podría activarla.	Las respuestas habladas son cortas y no empiezan con «Oye».	Vigilarlo en la prueba real.

En una frase

El modelo cumple las metas en el laboratorio. Su desempeño real con las voces, el acento y el ruido de *tu* cocina se sabrá con la fase 2b. Por eso está marcado como **beta**.

8. Integración en Quanela

Quién puede usarlo

Sigue el mismo modelo de funciones de IA y voz (ADR 0014):

usable = plataforma activa $\wedge$ incluida en el plan $\wedge$ la organización la ofrece $\wedge$ la cuenta la tiene activa $\wedge$ permiso de uso
y además: en este equipo alguien encendió «Manos libres»

Capa	Qué decide	Dónde
Plataforma	Encender o apagar la función para todos, y el umbral y los tramos por defecto	dk_features (voice_wake_word), panel de plataforma
Plan	Qué planes la incluyen: standard, business y enterprise, los mismos que los comandos de voz	dk_plan_features
Organización	Si la ofrece y en qué cuentas; puede ajustar el umbral si se lo permite	Administración de la organización → IA y voz
Cuenta y rol	La cocina la usa con kitchen.view y la configura con settings.manage	RBAC
Equipo	Abrir el micrófono en esta tableta (apagado por defecto)	IA y voz → Comandos de voz · En este equipo

Depende de voice_commands : la frase solo abre la escucha de comandos. Los ajustes se validan en la base: threshold entre 0,05 y 0,99 y confirm_frames de 1 a 5.

Lo que ve la gente

En ajustes

- El interruptor «**Manos libres (Oye Quanela)**» y su explicación de privacidad.
- Una barra de progreso durante la primera descarga.
- **Probar:** escucha la frase y muestra la probabilidad en vivo, la línea del umbral y cuántas veces detectó. Nunca abre un comando.

En Cocina

- Un indicador con una oreja: «Oye Quanela» (escuchando), «Preparando...», en espera mientras hay un comando, «En pausa» o error (por ejemplo, sin permiso de micrófono).
- **Un toque lo pausa o lo reanuda.**
- Al detectar: tono, y la burbuja del micrófono muestra lo que se escucha.

Privacidad

- El audio se procesa en memoria, tramo a tramo, **dentro del navegador**. No se guarda, no se envía y no hay servidores de voz.
- Solo se descargan los modelos, que son archivos públicos. Nada sube.
- El micrófono solo se abre en equipos donde alguien activó manos libres. El indicador es visible mientras escucha y se puede pausar.
- Lo que pasa *después* de la frase usa el motor de comandos elegido. Con el del navegador (Chrome), el comando de 5 s se procesa en servidores de Google, como hasta hoy con el botón del micrófono. Con Vosk, todo queda en el equipo.
- Las grabaciones para evaluar (fase 2b) requieren autorización escrita (Ley 1581). Se guardan fuera de la app y se borran al terminar.

Archivos principales

Archivo	Responsabilidad
<code>src/shared/voice/wakeWord/captureWorklet.js</code>	Captura y remuestreo a 16 kHz en tramos de 80 ms (hilo de audio)
<code>.../capture.ts</code>	Abre el micrófono (eco sí, supresión de ruido no) y conecta el worklet
<code>.../wakeWordStream.ts</code>	Cálculo por tramos y regla de detección (pura; con pruebas)
<code>.../wakeWordModel.ts</code>	Descarga de los 3 modelos con progreso y onnxruntime-web con carga diferida
<code>.../useWakeWord.ts</code>	Ciclo de vida: escuchar, suspender, pausar mientras habla Quanela, reiniciar tras detectar
<code>.../tuning.ts</code> , <code>preference.ts</code> , <code>tone.ts</code>	Umbral efectivo, interruptor por equipo y tono
<code>src/modules/kitchen/voice/WakeWordPanel.tsx</code> , <code>WakeWordIndicator.tsx</code>	Interfaz
<code>useVoiceCommandEngine.startHandsFree()</code>	Tono y ventana de 5 s
<code>supabase/migrations/20260930260000...</code> , <code>...270000...</code>	Función, planes y umbral 0,9

9. Rendimiento y costos

Recurso	Medido o estimado	Comentario
CPU por tramo de 80 ms	3,3–4,8 ms en un Mac M5 (4–6 % de un núcleo)	Medido en el navegador con carga alta del equipo. En una tableta modesta se estima de 5 a 10 veces más (20–50 % de un núcleo). Falta medirlo.
Hilo	WebAssembly en un solo hilo; el worklet va en el hilo de audio	No requiere encabezados especiales (COOP/COEP).
Memoria	Del orden de decenas de MB (runtime WASM y tres sesiones)	Solo en equipos con manos libres activo.
Descarga (una vez por equipo)	Modelos 3,26 MB y runtime 14,2 MB sin comprimir (3,7 MB gzip, ~2,3 MB brotli)	Los modelos tienen caché de 1 año en el CDN de Supabase; el runtime va con la app en Vercel.
Paquete principal de la app	Unos pocos KB de código	onnxruntime-web y el worklet están en archivos aparte que solo se descargan si se usa manos libres.
Costo en servidores por uso	Cero	No hay inferencia en servidor; solo la descarga estática de 3,3 MB por equipo.
Primera activación	~1–10 s de descarga y preparación; con caché, menos de 1 s	El indicador muestra «Preparando...».

10. Cómo escala

10.1 Más cocinas, más tabletas

La detección corre en cada equipo, así que el sistema escala de forma **lineal y sin costo marginal de cómputo**: 10 o 10.000 tabletas no cambian nada en los servidores.

Escenario	Tabletas	Descarga inicial total	Carga en servidores
Piloto	10	~55 MB	Ninguna
Crecimiento	1.000	~5,5 GB (una sola vez)	Ninguna; CDN
Escala	100.000	~550 GB (una sola vez)	Ninguna; CDN

La descarga se repite solo al publicar una versión nueva del modelo o si el navegador borra su caché.

10.2 Más frases

Como la base es compartida (sección 4), cada frase nueva es **otro clasificador de menos de 1 MB**. En el equipo, mel y características se calculan una sola vez por tramo, y cada clasificador extra cuesta décimas de milisegundo. Ejemplos:

- «Oye Quanela, pausa» o «Quanela, silencio» para controlar el propio sistema;
- una frase por marca, en esquemas de marca blanca («Oye Burger Lab»);
- frases de emergencia («Quanela, ayuda»).

Para cada frase se generan de 5.000 a 10.000 ejemplos sintéticos con los scripts existentes (`gen/texts.py` más los generadores), se reentrena y se evalúa. Es cuestión de horas de cómputo, no de semanas.

10.3 Más idiomas y acentos

- La red de características de Google no depende del idioma: se entrenó con audio muy variado. Lo que cambia son los datos del clasificador.
- Para portugués o inglés («Hey Quanela»), basta generar positivos con voces de ese idioma de licencia comercial (Kokoro tiene voces pt, en, fr e it) y usar habla de ese idioma como negativo.
- Para los acentos colombianos, lo que más rinde son las **grabaciones reales** de la fase 2b, más voces clonadas de hablantes colombianos con licencia comercial.

10.4 Personalización por organización o cocina

- **Hoy, sin código**: el umbral y los tramos se ajustan por plataforma, por organización o por cuenta (si la organización lo permite), con el sistema de ajustes por capas.
- **Siguiente paso posible**: un ajuste fino por cocina, con 50–100 frases grabadas en ese lugar (con consentimiento), para bajar las frases perdidas en su ruido y su acústica. Pesaría menos de 1 MB por cocina.

10.5 Escalar el entrenamiento

Etapas	Hoy (en un Mac)	Para escalar
Generar voz	Kokoro, ~100 clips por minuto con 4 procesos; Chatterbox, ~30 s por clip	Una GPU alquilada o más procesos: el trabajo está repartido por semillas y se puede paralelizar.
Características	~0,8 ms por ventana y fragmentos en paralelo	Más núcleos, u onnxruntime con GPU.
Entrenar	30–40 min por ronda en MPS	Una GPU lo lleva a minutos. El modelo es pequeño y no es el cuello de botella.
Evaluar	Caché de características de los clips de prueba (solo se clasifica)	Ya permite comparar decenas de variantes rápidamente.

10.6 Techo y alternativas

Si la prueba real mostrara que no alcanza, hay dos caminos:

- **Mejorar el propio modelo:** datos reales, clasificador temporal (GRU o convolución) en vez de denso, más negativos reales.
- **Usar Picovoice Porcupine:** comercial, con soporte oficial en español, pero con licencia de pago y dependencia de un proveedor.

La arquitectura de la app lo permite, porque el detector está detrás de `useWakeWord`.

11. Operación

11.1 Ajustar la sensibilidad (sin publicar código)

Con el panel de plataforma o con el RPC `dk_platform_set_feature('voice_wake_word', p_default_settings => ...)`:

Síntoma en cocinas reales	Acción
Se activa solo, sin que nadie lo llame	Subir <code>threshold</code> (0,9 → 0,95 o 0,97) o <code>confirm_frames</code> (2 → 3)
Hay que repetir la frase a menudo	Bajar <code>threshold</code> (0,9 → 0,8); las falsas se mantuvieron en 0 en la evaluación
Solo una cocina tiene problemas	Ajustarlo para esa organización o cuenta (ajustes por capas)

11.2 Publicar una versión nueva del modelo

1. Entrenar y evaluar con `ml/wake-word` (README). Comparar con el anexo A sobre el **mismo** conjunto de evaluación.
2. Subir los archivos a una carpeta **nueva**, por ejemplo `dk-voice-models/wake/oye-quanela-v2/`, con caché de 1 año. Nunca se sobrescribe una versión publicada, porque su caché es inmutable.
3. Cambiar `WAKE_WORD_VERSION` en `wakeWordModel.ts` y desplegar la app. Cada equipo descarga la nueva versión la próxima vez que active manos libres.
4. Si el umbral óptimo cambió, agregar una migración que actualice `default_settings`.

11.3 Revertir

Hay tres niveles, del más rápido al más lento:

1. apagar la función desde la plataforma (efecto inmediato para todos);
2. subir el umbral;
3. volver `WAKE_WORD_VERSION` a la versión anterior (sigue en el bucket) y desplegar.

11.4 Diagnosticar un problema en una tableta

- **Ajustes** → **Probar**: muestra la probabilidad en vivo. Si al decir la frase no pasa la línea del umbral, es un problema de audio (distancia, ruido o micrófono). Si sube pero no llega, bajar el umbral para esa cuenta.
- **Indicador en error**: casi siempre es el permiso del micrófono o un navegador sin soporte.
- **Se corta solo en iOS**: la pantalla se bloqueó. Dejarla encendida.
- **Hoy no hay telemetría** de activaciones. Propuesta: contar en forma anónima las activaciones, los comandos válidos tras activar y las «activaciones sin comando». No requiere guardar audio y ayudaría a detectar falsas activaciones en producción.

12. Cómo mejorarlo: hoja de ruta

Prioridad	Acción	Beneficio esperado	Esfuerzo
1	Fase 2b : 10–20 personas × 10 frases y 1–2 h de cocina en servicio, con consentimiento. Se evalúa con <code>python -m ww.evaluate_real</code> .	Cifras reales; confirmar o ajustar el umbral.	1–2 días de grabación y 1 hora de análisis
2	Medir CPU, temperatura y batería en la tableta objetivo durante un turno.	Saber si sirve para un turno completo.	Medio día
3	Reentrenar con el audio real de la cocina: sus ruidos como negativos y fondos, y las frases reales como positivos si hay consentimiento.	Menos frases perdidas con ruido y menos falsas.	1 día
4	Telemetría anónima de activaciones (sin audio).	Detectar problemas en producción por cocina.	1–2 días
5	Clasificador temporal (GRU) y más frases dirigidas.	Menos frases parecidas que activan.	1–2 días
6	Frases adicionales («pausa», «silencio») u otros idiomas.	Más control por voz.	~1 día por frase
7	Actualizar el manual de usuario, solo cuando se pida.	Adopción.	Bajo

13. Licencias y cumplimiento

El criterio fue usar **solo fuentes que permiten uso comercial**, verificadas una por una. Donde había duda, se excluyó la fuente.

Componente	Licencia	Uso
Arquitectura y código de referencia de openWakeWord	Apache 2.0	Referencia; nuestro código es propio
Google <code>speech_embedding/1</code>	Apache 2.0	Pesos de la red de características
onnxruntime-web	MIT	Ejecución en el navegador
Kokoro-82M	Apache 2.0	Voz sintética para entrenar
Piper <code>es_ES-carlrm-x_low</code>	Datos de dominio público, entrenada desde cero	Voz sintética para entrenar
Chatterbox Multilingual	MIT	Voces clonadas, solo para evaluar
FLEURS <code>es_419</code>	CC BY 4.0 (atribución)	Habla negativa y referencias para clonar
MUSAN	Creative Commons sin uso no comercial, o dominio público	Ruido, música y habla
OpenSLR 28: <code>simulated_rirs</code> y <code>pointsource_noises</code>	Apache 2.0	Reverberación y ruidos

Excluido a propósito

- Los modelos preentrenados de openWakeWord (CC BY-NC-SA) y el conjunto ACAV100M.
- Las otras 7 voces de Piper en español: se ajustaron a partir de voces inglesas con licencia de investigación (Lessac) o no comercial (Ryan).
- XTTS (no comercial) y las voces del sistema de Apple.
- Las respuestas de sala reales de OpenSLR 28 (RWCP, AIR y REVERB tienen otras licencias).
- Common Voice, que ahora exige cuenta y aceptar términos. Se reemplazó por FLEURS.

Atribución requerida: FLEURS (Google, CC BY 4.0) y MUSAN (Snyder, Chen y Povey, 2015) se citan en `ml/wake-word/README.md`. Los modelos publicados no contienen audio de esas fuentes, solo pesos aprendidos.

Datos personales: el sistema no recolecta audio. Las grabaciones de la fase 2b requieren autorización escrita según la Ley 1581 de 2012, se guardan fuera de la app y se borran al terminar, salvo que se autorice conservarlas.

Anexo A. Tabla completa de umbrales (modelo publicado)

502 frases apartadas (400 de Kokoro y 102 clonadas de hablantes reales) en tres condiciones; 1.600 intentos de frases parecidas; 17,3 h de audio sin la frase. La latencia se mide respecto del final del clip, incluida la cola de reverberación.

Umbral	Tramos	Perdidas sin ruido	10 dB	5 dB	Falsas/h	Parecidas (de 1.600)	Latencia mediana	p90
0,30	1	0,2 %	0,2 %	1,6 %	0,81	34	−220 ms	−38 ms
0,30	2	0,2 %	0,6 %	2,6 %	0,29	11	−140 ms	43 ms
0,30	3	0,2 %	0,6 %	4,0 %	0,12	6	−60 ms	126 ms
0,50	1	0,2 %	0,4 %	2,0 %	0,58	22	−214 ms	−30 ms
0,50	2	0,2 %	0,6 %	2,8 %	0,12	10	−134 ms	51 ms
0,50	3	0,2 %	0,6 %	4,4 %	0,00	6	−53 ms	133 ms
0,70	1	0,2 %	0,6 %	2,0 %	0,29	19	−206 ms	−29 ms
0,70	2	0,2 %	0,8 %	3,0 %	0,00	8	−126 ms	53 ms
0,70	3	0,4 %	1,0 %	4,8 %	0,00	5	−43 ms	133 ms
0,80	1	0,2 %	0,6 %	2,4 %	0,23	16	−200 ms	−26 ms
0,80	2	0,2 %	1,2 %	3,8 %	0,00	8	−120 ms	54 ms
0,80	3	0,4 %	1,2 %	6,2 %	0,00	3	−40 ms	134 ms
0,90	1	0,2 %	0,6 %	3,0 %	0,17	13	−200 ms	−20 ms
0,90	2	0,4 %	1,2 %	4,4 %	0,00	4	−119 ms	60 ms
0,90	3	0,4 %	2,6 %	6,8 %	0,00	2	−39 ms	140 ms
0,95	1	0,2 %	0,8 %	3,6 %	0,12	11	−190 ms	−20 ms
0,95	2	0,4 %	2,0 %	6,2 %	0,00	4	−110 ms	60 ms
0,95	3	1,0 %	3,6 %	8,2 %	0,00	1	−30 ms	140 ms
0,97	2	0,6 %	2,2 %	6,4 %	0,00	3	−110 ms	64 ms
0,99	1	0,4 %	2,4 %	5,8 %	0,00	4	−175 ms	0 ms
0,99	2	1,4 %	3,6 %	8,8 %	0,00	1	−97 ms	80 ms
0,99	3	2,0 %	5,4 %	12,0 %	0,00	1	−19 ms	158 ms

Anexo B. Archivos, comandos y glosario

Fuentes de este documento: ADR 0016 (sección 10), `ml/wake-word/README.md` , los resultados de evaluación del modelo publicado (ronda 4) y las mediciones en el navegador del 30 de septiembre de 2026.

Modelos publicados (`dk-voice-models/wake/oye-quanela-v1/`)

Archivo	Bytes	SHA-256
<code>meIspectrogram.onnx</code>	1.086.693	<code>23a9aada0160ee63c37dfe9c076b51049f12cb49dac66750495793bba95a14b9</code>
<code>speech_embedding.onnx</code>	1.316.738	<code>7ddc8dd0828196507ea5f64e7e336366e16187676f3bcc3c6306f9fa1f3be96e</code>
<code>oye_quanela.onnx</code>	859.926	<code>cce08899e6a0a73f4240a929d5188b0f0ee60baaf9c7b35f35bac284476208ae</code>

Comandos útiles (desde `ml/wake-word/`)

```
# Modelos base
python build_embedding.py && python build_melspec.py
# Voz sintética (KIND: pos | adv | hard | kitchen | speech)
python gen/gen_kokoro.py pos 1500 train 11
# Características (en fragmentos paralelos) y audio largo
python -m ww.build clips train pos 3 300 0/4
python -m ww.build streams train
# Entrenar, exportar y evaluar
WW_DROPOUT=0.3 WW_FEAT_NOISE=0.3 WW_WD=0.05 python -m ww.train models/final 10000
python -m ww.export models/final
python -m ww.evaluate models/final
# Grabaciones reales (fase 2b)
python -m ww.evaluate_real models/final CARPETA_FRASES CARPETA_AMBIENTE 0.9 2
```

Glosario

Palabra de activación	Frase fija que el equipo escucha de forma continua para empezar a atender («Oye Quanela»).
Tramo	80 ms de audio (1.280 muestras a 16 kHz). El detector da una respuesta por tramo.
Espectrograma mel	Representación del sonido por bandas de frecuencia en una escala parecida a la del oído.
Umbral	Probabilidad mínima (0–1) para contar un tramo como «Sí».
Tramos de confirmación	Cuántos tramos seguidos deben superar el umbral para activar.
Activación perdida	Se dijo la frase y no se activó (en inglés, <i>false reject</i>).
Activación falsa	Se activó sin que se dijera la frase (<i>false accept</i>). Se mide por hora de audio.
Relación señal/ruido (dB)	Cuánto más fuerte que el ruido está la voz: 10 dB es ruido moderado y 5 dB, fuerte.
Aumento de datos	Variaciones artificiales (ruido, sala, velocidad) para que el modelo aprenda condiciones reales.
Negativo difícil	Audio sin la frase que el modelo confunde, y que se usa para reentrenarlo.
ONNX / onnxruntime-web	Formato estándar de modelos y el motor que los ejecuta en el navegador con WebAssembly.
AudioWorklet	Código que el navegador ejecuta en el hilo de audio, para procesar el micrófono sin cortes.
Streaming	Procesar el audio a medida que llega, tramo a tramo, en vez de un archivo completo.